

Цель работы

Добавить в приложение модуль расчёта статистики на основе хранимых данных и создать пользовательский интерфейс для отображения аналитической информации.

Задачи

4.1 Определение метрик и создание классов для статистики

- Открыть ТЗ и определить 2-3 метрики для построения диаграмм
- Создать в проекте `<ИмяПроекта>.Domain` папку `Statistics`
- Создать record-классы для хранения данных статистики

Примечание: Создайте 2-3 record-класса на основе выбранных метрик. Используйте шаблоны ниже, адаптируя их под свою предметную область.

Шаблон 1: Статистика по enum (для круговой диаграммы)

Файл: `<ИмяПроекта>.Domain/Statistics/<Тип>StatisticItem.cs`

```
namespace <ИмяПроекта>.Domain.Statistics;

public record <Тип>StatisticItem
{
    public required <ВашEnum> <СвойствоEnum> { get; set; }
    public required int Count { get; set; }
}
```

Конкретный пример для салона красоты:

```
namespace BeautyDB.Domain.Statistics;

public record StatusStatisticItem
{
    public required ReservationStatus Status { get; set; }
    public required int Count { get; set; }
}
```

Шаблон 2: Статистика по месяцам (для линейной диаграммы)

```
namespace <ИмяПроекта>.Domain.Statistics;

public record MonthStatisticItem
{
    public required int Year { get; set; }
    public required int Month { get; set; }
    public required int Count { get; set; }

    public string GetMonthName()
    {
        var date = new DateTime(Year, Month, 1);
        return date.ToString("MMM yyyy");
    }
}
```

Примечание: Этот класс универсален для любой предметной области с датами.

Шаблон 3: Статистика по связанной сущности (для столбчатой диаграммы)

Файл: <ИмяПроекта>.Domain/Statistics/<СвязаннаяСущность>StatisticItem.cs

```
namespace <ИмяПроекта>.Domain.Statistics;

public record <СвязаннаяСущность>StatisticItem
{
    public required string <СвязаннаяСущность>Name { get; set; }
    public required int Count { get; set; }
}
```

Конкретный пример для салона красоты:

```
namespace BeautyDB.Domain.Statistics;

public record MasterStatisticItem
{
    public required string MasterName { get; set; }
    public required int Count { get; set; }
}
```

Подсказка: Выберите метрики на основе полей ваших классов. Например: группировка по статусам, по типам, по периодам времени, по исполнителям.

4.2 Создание сервиса статистики

- Создать проект <ИмяПроекта>.Services для бизнес-логики
- Создать заготовку класса StatisticsService (детальная реализация методов будет в шаге 4.4)
- Добавить ссылки между проектами:
 - <ИмяПроекта>.Services → <ИмяПроекта>.Domain
 - <ИмяПроекта>.Services → <ИмяПроекта>.Data.Interfaces
 - <ИмяПроекта>.UI → <ИмяПроекта>.Services

Ожидаемая структура проекта после создания сервиса:

```
Solution/
├── <ИмяПроекта>.UI/           # WPF формы и элементы управления
├── <ИмяПроекта>.Domain/       # Классы предметной области + модели статистики
├── <ИмяПроекта>.Services/     # Бизнес-логика и сервисы (Новое)
├── <ИмяПроекта>.Data.Interfaces/ # Интерфейсы репозиторий
└── <ИмяПроекта>.Data.InMemory/ # In-memory реализация хранения данных
```

4.3 Добавление фильтрации в репозиторий

Для фильтрации данных создадим класс фильтра и изменим метод GetAll .

Шаг 1: Создать класс фильтра в проекте <ИмяПроекта>.Data.Interfaces

Шаблон:

Файл: <ИмяПроекта>.Data.Interfaces/<Сущность>Filter.cs

```
namespace <ИмяПроекта>.Data.Interfaces;

public record <Сущность>Filter
```

```

{
    public static <Сущность>Filter Empty => new();

    // Если у вашей сущности есть свойство с датой:
    public DateOnly? StartDate { get; init; }
    public DateOnly? EndDate { get; init; }

    // Добавьте другие параметры фильтрации на основе ваших полей
}

```

Конкретный пример для салона красоты:

```

namespace BeautyDB.Data.Interfaces;

public record ReservationsFilter
{
    public static ReservationsFilter Empty => new();

    public DateOnly? StartDate { get; init; }
    public DateOnly? EndDate { get; init; }
}

```

Шаг 2: Изменить сигнатуру метода GetAll в интерфейсе репозитория

Шаблон:

```

public interface I<Сущность>Repository
{
    List<<Сущность>> GetAll(<Сущность>Filter filter);
    // ... остальные методы
}

```

Конкретный пример:

```

namespace BeautyDB.Data.Interfaces;

using BeautyDB.Domain;

public interface IReservationRepository
{
    List<Reservation> GetAll(ReservationsFilter filter);
    // ... остальные методы
}

```

Шаг 3: Реализовать фильтрацию в in-memory репозитории

Шаблон:

```

public List<<Сущность>> GetAll(<Сущность>Filter filter)
{
    var result = _allItems.AsEnumerable();

    if (filter.StartDate.HasValue)
        result = result.Where(x => x.<СвойствоСДатой> >= filter.StartDate.Value);

    if (filter.EndDate.HasValue)
        result = result.Where(x => x.<СвойствоСДатой> <= filter.EndDate.Value);
}

```

```
return result.ToList();  
}
```

Конкретный пример:

```
namespace BeautyDB.Data.InMemory;  
  
public class ReservationRepository : IReservationRepository  
{  
    private readonly List<Reservation> _reservations = new();  
  
    public List<Reservation> GetAll(ReservationsFilter filter)  
    {  
        var result = _reservations.AsEnumerable();  
  
        if (filter.StartDate.HasValue)  
            result = result.Where(r => r.Date >= filter.StartDate.Value);  
  
        if (filter.EndDate.HasValue)  
            result = result.Where(r => r.Date <= filter.EndDate.Value);  
  
        return result.ToList();  
    }  
  
    // ... остальные методы  
}
```

Преимущества такого подхода:

- Легко расширять новыми параметрами фильтрации без изменения сигнатуры метода
- Все параметры в одном месте

4.4 Реализация методов расчёта статистики

Добавьте в класс `StatisticsService` (созданный в шаге 4.2) методы, которые возвращают списки элементов статистики. Для каждой метрики создайте отдельный метод.

Примечание: Используйте шаблоны ниже, адаптируя их под свои сущности и свойства.

Метод 1: Статистика по еnum (группировка)

Шаблон:

```
public List<<Тип>StatisticItem> GetBy<Свойство>(<Сущность>Filter filter)  
{  
    var items = _repository.GetAll(filter);  
  
    return items  
        .GroupBy(x => x.<СвойствоEnum>)  
        .Select(g => new <Тип>StatisticItem  
        {  
            <СвойствоEnum> = g.Key,  
            Count = g.Count()  
        })  
        .OrderBy(s => s.<СвойствоEnum>)  
        .ToList();  
}
```

Конкретный пример для статусов резерваций:

```

public List<StatusStatisticItem> GetReservationsByStatus(ReservationsFilter filter)
{
    var reservations = _reservationRepository.GetAll(filter);

    return reservations
        .GroupBy(r => r.Status)
        .Select(g => new StatusStatisticItem
        {
            Status = g.Key,
            Count = g.Count()
        })
        .OrderBy(s => s.Status)
        .ToList();
}

```

Метод 2: Статистика по месяцам (динамика)

Шаблон (универсальный для сущностей с датой):

```

public List<MonthStatisticItem> GetByMonth(<Сущность>Filter filter)
{
    var items = _repository.GetAll(filter);

    return items
        .GroupBy(x => new { x.<СвойствоСДатой>.Year, x.<СвойствоСДатой>.Month })
        .Select(g => new MonthStatisticItem
        {
            Year = g.Key.Year,
            Month = g.Key.Month,
            Count = g.Count()
        })
        .OrderBy(m => m.Year)
        .ThenBy(m => m.Month)
        .ToList();
}

```

Конкретный пример для резерваций:

```

public List<MonthStatisticItem> GetReservationsByMonth(ReservationsFilter filter)
{
    var reservations = _reservationRepository.GetAll(filter);

    return reservations
        .GroupBy(r => new { r.Date.Year, r.Date.Month })
        .Select(g => new MonthStatisticItem
        {
            Year = g.Key.Year,
            Month = g.Key.Month,
            Count = g.Count()
        })
        .OrderBy(m => m.Year)
        .ThenBy(m => m.Month)
        .ToList();
}

```

Метод 3: Статистика по связанной сущности (рейтинг)

Шаблон:

```

public List<<СвязаннаяСущность>StatisticItem> GetBy<СвязаннаяСущность>(<Сущность>Filter filter)
{
    var items = _repository.GetAll(filter);

    return items
        .Where(x => x.<СвязаннаяСущность> != null)
        .GroupBy(x => x.<СвязаннаяСущность>!.Name)
        .Select(g => new <СвязаннаяСущность>StatisticItem
        {
            <СвязаннаяСущность>Name = g.Key,
            Count = g.Count()
        })
        .OrderByDescending(s => s.Count)
        .ToList();
}

```

Конкретный пример для мастеров салона:

```

public List<MasterStatisticItem> GetReservationsByMaster(ReservationsFilter filter)
{
    var reservations = _reservationRepository.GetAll(filter);

    return reservations
        .Where(r => r.Master != null)
        .GroupBy(r => r.Master!.Name)
        .Select(g => new MasterStatisticItem
        {
            MasterName = g.Key,
            Count = g.Count()
        })
        .OrderByDescending(m => m.Count)
        .ToList();
}

```

Важные моменты:

- Используйте LINQ для группировки и агрегации
- Сортируйте результаты для удобства отображения

Полный пример структуры `StatisticsService.cs` для салона красоты:

```

namespace BeautyDB.Services;

using BeautyDB.Data.Interfaces;
using BeautyDB.Domain.Statistics;

/// <summary>
/// Сервис для расчёта статистики по бронированиям
/// </summary>
public class StatisticsService
{
    private readonly IReservationRepository _reservationRepository;

    public StatisticsService(IReservationRepository reservationRepository)
    {
        _reservationRepository = reservationRepository;
    }

    /// <summary>

```

```

/// Получить распределение бронирований по статусам
/// </summary>
public List<StatusStatisticItem> GetReservationsByStatus(ReservationsFilter filter)
{
    var reservations = _reservationRepository.GetAll(filter);

    return reservations
        .GroupBy(r => r.Status)
        .Select(g => new StatusStatisticItem
        {
            Status = g.Key,
            Count = g.Count()
        })
        .OrderBy(s => s.Status)
        .ToList();
}

/// <summary>
/// Получить динамику бронирований по месяцам
/// </summary>
public List<MonthStatisticItem> GetReservationsByMonth(ReservationsFilter filter)
{
    var reservations = _reservationRepository.GetAll(filter);

    return reservations
        .GroupBy(r => new { r.Date.Year, r.Date.Month })
        .Select(g => new MonthStatisticItem
        {
            Year = g.Key.Year,
            Month = g.Key.Month,
            Count = g.Count()
        })
        .OrderBy(m => m.Year)
        .ThenBy(m => m.Month)
        .ToList();
}

/// <summary>
/// Получить статистику по популярности мастеров
/// </summary>
public List<MasterStatisticItem> GetReservationsByMaster(ReservationsFilter filter)
{
    var reservations = _reservationRepository.GetAll(filter);

    return reservations
        .Where(r => r.Master != null)
        .GroupBy(r => r.Master!.Name)
        .Select(g => new MasterStatisticItem
        {
            MasterName = g.Key,
            Count = g.Count()
        })
        .OrderByDescending(m => m.Count)
        .ToList();
}
}

```

Важные моменты:

- Используйте LINQ для группировки и агрегации
- Сортируйте результаты для удобства отображения

4.5 Создание формы статистики

- Добавить новую форму `StatisticsWindow` в проект `<ИмяПроекта>.UI`
- Разместить элементы:
 - Фильтры
 - Кнопки "Применить фильтр" и "Сбросить"
 - `TabControl` для организации графиков по вкладкам

Пример структуры XAML:

```
<Window x:Class="BeautyDB.UI.StatisticsWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:converters="clr-namespace:BeautyDB.UI.Converters"
        Title="Статистика записей" Height="800" Width="1200">
    <Window.Resources>
        <converters:DateOnlyConverter x:Key="DateOnlyConverter"/>
    </Window.Resources>
    <Grid Margin="10">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*/>
        </Grid.RowDefinitions>

        <!-- Заголовок -->
        <TextBlock Grid.Row="0" Text="Статистика записей"
                  FontSize="24" FontWeight="Bold" Margin="0,0,0,10"/>

        <!-- Панель фильтрации -->
        <StackPanel Grid.Row="1" Orientation="Horizontal" Margin="0,0,0,15">
            <TextBlock Text="Период с:" VerticalAlignment="Center" Margin="0,0,5,0"/>
            <DatePicker SelectedDate="{Binding StartDate, Converter={StaticResource
DateOnlyConverter}, ConverterParameter=DateTime}"
                      Width="120" Margin="0,0,15,0"/>
            <TextBlock Text="по:" VerticalAlignment="Center" Margin="0,0,5,0"/>
            <DatePicker SelectedDate="{Binding EndDate, Converter={StaticResource
DateOnlyConverter}, ConverterParameter=DateTime}"
                      Width="120" Margin="0,0,15,0"/>
            <Button Content="Применить фильтр" Width="130" Height="30"
                  Click="ApplyFilterButton_Click"/>
            <Button Content="Сбросить" Width="100" Height="30"
                  Margin="10,0,0,0" Click="ResetFilterButton_Click"/>
        </StackPanel>

        <!-- Контейнер для диаграмм -->
        <TabControl Grid.Row="2">
            <!-- Вкладки с графиками будут добавлены после настройки OxyPlot -->
        </TabControl>
    </Grid>
</Window>
```

Пример code-behind `StatisticsWindow.xaml.cs`:

```
namespace BeautyDB.UI;

using System.Windows;

public partial class StatisticsWindow : Window
{
```



```

public DateOnly? StartDate { get; set; }
public DateOnly? EndDate { get; set; }

public StatisticsWindow()
{
    InitializeComponent();
    DataContext = this;
}

private void ApplyFilterButton_Click(object sender, RoutedEventArgs e)
{
    // Загрузка статистики с фильтрацией
}

private void ResetFilterButton_Click(object sender, RoutedEventArgs e)
{
    StartDate = null;
    EndDate = null;
    // Загрузка статистики без фильтров
}
}

```

4.6 Подключение и настройка OxyPlot

Установка пакета

1. Открыть проект <ИмяПроекта>.UI в Solution Explorer
2. Щёлкнуть правой кнопкой → **Manage NuGet Packages**
3. Перейти на вкладку **Browse**
4. Найти и установить пакет: **OxyPlot.Wpf** (версия 2.2.0 или новее)

Настройка в XAML

Добавить пространство имён в окно:

```
xmlns:oxy="http://oxyplot.org/wpf"
```

Полный пример заголовка Window:

```

<Window x:Class="BeautyDB.UI.StatisticsWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:oxy="http://oxyplot.org/wpf"
    xmlns:converters="clr-namespace:BeautyDB.UI.Converters"
    Title="Статистика записей" Height="800" Width="1200">

```

4.7 Визуализация данных с помощью OxyPlot

Общая концепция построения диаграмм:

1. **PlotModel** — создаём модель диаграммы
2. **Series** — создаём серию данных нужного типа (PieSeries, BarSeries, LineSeries)
3. **Заполнение данными** — получаем данные из сервиса и добавляем в серию
4. **Привязка к PlotView** — сохраняем модель в свойство, которое привязано к PlotView в XAML

Пример 1: Круговая диаграмма (PieSeries)

Применение: Распределение по категориям (enum, типы, статусы).

Что адаптировать под свою предметную область:

- Название метода загрузки (Load...Chart)
- Вызов метода сервиса (GetBy...)
- Получение текста для меток (конвертер для enum или прямое использование строк)
- Название свойства PlotModel

XAML:

```
<TabItem Header="По статусам">
    <oxy:PlotView Model="{Binding StatusPlotModel}" Margin="10"/>
</TabItem>
```

C# код:

```
using OxyPlot;
using OxyPlot.Series;
using BeautyDB.UI.Converters;

// Свойство для хранения модели (привязывается к PlotView в XAML)
public PlotModel? StatusPlotModel { get; set; }

private void LoadStatusChart(ReservationsFilter filter)
{
    // 1. Получаем данные из сервиса статистики
    var data = _statisticsService.GetReservationsByStatus(filter);

    // 2. Создаём модель диаграммы
    var plotModel = new PlotModel { Title = "" };

    // 3. Создаём серию для круговой диаграммы
    var pieSeries = new PieSeries
    {
        StrokeThickness = 2.0,
        InsideLabelPosition = 0.5,
        AngleSpan = 360,
        StartAngle = 0
    };

    // 4. Заполняем данными
    foreach (var item in data)
    {
        // Для enum используем конвертер для получения русского текста
        var label = ReservationStatusConverter.GetStatusText(item.Status);
        // Для строковых свойств используйте напрямую: var label = item.SomeName;

        pieSeries.Slices.Add(new PieSlice(label, item.Count)
        {
            IsExploded = false // true = сегмент "выдвинут" из круга
        });
    }

    // 5. Добавляем серию в модель
    plotModel.Series.Add(pieSeries);

    // 6. Сохраняем модель в свойство (автоматически отобразится в PlotView)
    StatusPlotModel = plotModel;
}
```

Ключевые свойства PieSeries:

- `StrokeThickness` - толщина границы сегментов
- `InsideLabelPosition` - позиция подписей (0.0 - центр, 1.0 - край)
- `IsExploded` - "выдвинуть" сегмент из круга для акцента

Пример 2: Столбчатая диаграмма (BarSeries)

Применение: Рейтинги, сравнение показателей по связанным сущностям.

Что адаптировать под свою предметную область:

- Название метода загрузки (`Load...Chart`)
- Вызов метода сервиса (`GetBy...`)
- Названия осей и их заголовки
- Получение меток для оси категорий (`.Name` или другое свойство)
- Название свойства `PlotModel`

Особенность: Для столбчатой диаграммы нужно создать **две оси** — категорий и значений.

XAML:

```
<TabItem Header="По мастерам">
    <oxy:PlotView Model="{Binding MasterPlotModel}" Margin="10"/>
</TabItem>
```

C# код:

```
using OxyPlot;
using OxyPlot.Axes;
using OxyPlot.Series;

public PlotModel? MasterPlotModel { get; set; }

private void LoadMasterChart(ReservationsFilter filter)
{
    // 1. Получаем данные из сервиса
    var data = _statisticsService.GetReservationsByMaster(filter);

    // 2. Создаём модель диаграммы
    var plotModel = new PlotModel { Title = "" };

    // 3. Создаём ось категорий (для текстовых меток)
    var categoryAxis = new CategoryAxis
    {
        Position = AxisPosition.Left, // Слева для горизонтальных столбцов
        Title = "Мастера"
    };

    // 4. Заполняем метки оси (важно: в том же порядке, что и данные!)
    foreach (var item in data)
    {
        categoryAxis.Labels.Add(item.MasterName);
    }

    plotModel.Axes.Add(categoryAxis);

    // 5. Создаём числовую ось (для значений)
    plotModel.Axes.Add(new LinearAxis
    {
        Position = AxisPosition.Bottom, // Снизу для значений
        Title = "Количество записей",
```

```

        MinimumPadding = 0.1,
        MaximumPadding = 0.1
    });

    // 6. Создаём серию столбцов
    var barSeries = new BarSeries
    {
        Title = "Количество записей",
        FillColor = OxyColor.FromRgb(79, 129, 189) // Цвет столбцов
    };

    // 7. Заполняем столбцы данными (порядок должен совпадать с Labels)
    foreach (var item in data)
    {
        barSeries.Items.Add(newBarItem { Value = item.Count });
    }

    plotModel.Series.Add(barSeries);
    MasterPlotModel = plotModel;
}

```

Ключевые моменты:

- `CategoryAxis` — ось с текстовыми метками (названия категорий)
- `LinearAxis` — ось с числовыми значениями
- `Position` — расположение оси (Left, Right, Top, Bottom)
- **ВАЖНО:** Порядок `Labels` в `CategoryAxis` должен точно соответствовать порядку `BarItem`
- Для вертикальных столбцов используйте `ColumnSeries` вместо `BarSeries`

Пример 3: Линейный график (`LineSeries`)

Применение: Отображение динамики показателей во времени (месяцы, недели, годы).

Что адаптировать под свою предметную область:

- Название метода загрузки (`Load...Chart`)
- Вызов метода сервиса (`GetByMonth`)
- Названия осей
- Метод получения меток для временной оси (`GetMonthName()` или аналогичный)
- Название свойства `PlotModel`

Особенность: Используется `DataPoint(x, y)`, где `x` — индекс периода, `y` — значение.

XAML:

```

<TabItem Header="По месяцам">
    <oxy:PlotView Model="{Binding MonthPlotModel}" Margin="10"/>
</TabItem>

```

C# код:

```

using OxyPlot;
using OxyPlot.Axes;
using OxyPlot.Series;

public PlotModel? MonthPlotModel { get; set; }

private void LoadMonthChart(ReservationsFilter filter)
{

```

```

// 1. Получаем данные из сервиса (данные уже отсортированы по году и месяцу)
var data = _statisticsService.GetReservationsByMonth(filter);

// 2. Создаём модель диаграммы
var plotModel = new PlotModel { Title = "" };

// 3. Создаём ось времени (категорий) снизу
var categoryAxis = new CategoryAxis
{
    Position = AxisPosition.Bottom,
    Angle = -15, // Поворот меток для лучшей читаемости
    Title = "Месяцы"
};

// 4. Заполняем метки оси
foreach (var item in data)
{
    categoryAxis.Labels.Add(item.GetMonthName()); // "Янв 2025", "Фев 2025"...
}

plotModel.Axes.Add(categoryAxis);

// 5. Создаём числовую ось (значений) слева
plotModel.Axes.Add(new LinearAxis
{
    Position = AxisPosition.Left,
    Title = "Количество записей",
    MinimumPadding = 0.1,
    MaximumPadding = 0.1
});

// 6. Создаём серию линий
var lineSeries = new LineSeries
{
    Title = "Количество записей",
    Color = OxyColor.FromRgb(79, 129, 189),
    MarkerType = MarkerType.Circle, // Форма маркеров на точках
    MarkerSize = 4,
    MarkerFill = OxyColor.FromRgb(79, 129, 189)
};

// 7. Добавляем точки на график (x = индекс, y = значение)
for (int i = 0; i < data.Count; i++)
{
    lineSeries.Points.Add(new DataPoint(i, data[i].Count));
}

plotModel.Series.Add(lineSeries);
MonthPlotModel = plotModel;
}

```

Ключевые моменты:

- `CategoryAxis` с `Angle = -15` поворачивает метки для лучшей читаемости
- `DataPoint(x, y)` — точка на графике, где `x` — индекс в массиве, `y` — значение
- `MarkerType` — форма маркеров (`Circle`, `Square`, `Diamond`, `Triangle`, `None`)
- `MarkerSize` — размер маркеров в пикселях
- `MarkerFill` — цвет заливки маркеров
- `Color` — цвет линии графика
- **Важно:** Данные должны быть отсортированы по времени (это делается в сервисе)

4.7.5 Обновление диаграмм через INotifyPropertyChanged

При привязке данных в WPF возникает проблема: когда свойство `PlotModel` получает новое значение, интерфейс не обновляется автоматически. WPF не знает об изменении свойства, так как класс не уведомляет об этом.

Решение — реализовать интерфейс `INotifyPropertyChanged`, который позволяет классу сообщать WPF об изменениях свойств. После этого все привязки будут обновляться автоматически.

Пример:

```
using System.ComponentModel;
using System.Runtime.CompilerServices;
using System.Windows;
using OxyPlot;

namespace BeautyDB.UI;

public partial class StatisticsWindow : Window, INotifyPropertyChanged
{
    private readonly StatisticsService _statisticsService;

    // Backing fields для свойств
    private DateOnly? _startDate;
    private DateOnly? _endDate;
    private PlotModel? _statusPlotModel;
    private PlotModel? _masterPlotModel;
    private PlotModel? _monthPlotModel;

    // Свойства с уведомлением
    public DateOnly? StartDate
    {
        get => _startDate;
        set
        {
            _startDate = value;
            OnPropertyChanged();
        }
    }

    public DateOnly? EndDate
    {
        get => _endDate;
        set
        {
            _endDate = value;
            OnPropertyChanged();
        }
    }

    public PlotModel? StatusPlotModel
    {
        get => _statusPlotModel;
        set
        {
            _statusPlotModel = value;
            OnPropertyChanged();
        }
    }

    public PlotModel? MasterPlotModel
```

```

{
    get => _masterPlotModel;
    set
    {
        _masterPlotModel = value;
        OnPropertyChanged();
    }
}

public PlotModel? MonthPlotModel
{
    get => _monthPlotModel;
    set
    {
        _monthPlotModel = value;
        OnPropertyChanged();
    }
}

// Реализация интерфейса
public event PropertyChangedEventHandler? PropertyChanged;

protected void OnPropertyChanged([CallerMemberName] string? propertyName = null)
{
    PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
}

// Конструктор и методы без изменений
public StatisticsWindow(StatisticsService statisticsService)
{
    InitializeComponent();
    _statisticsService = statisticsService;
    DataContext = this;
    LoadStatistics();
}

// ...
}

```

Примечания:

- Backing fields именуются с префиксом `_` в camelCase
- Атрибут `[CallerMemberName]` автоматически подставляет имя свойства при вызове `OnPropertyChanged()`
- Свойства только для чтения не требуют реализации уведомлений
- После этих изменений диаграммы будут обновляться автоматически при изменении фильтров

4.8 Интеграция статистики в приложение

Шаг 1: Дополнить code-behind StatisticsWindow

```

namespace BeautyDB.UI;

using System.ComponentModel;
using System.Runtime.CompilerServices;
using System.Windows;
using BeautyDB.Data.Interfaces;
using BeautyDB.Services;
using OxyPlot;

public partial class StatisticsWindow : Window, INotifyPropertyChanged

```

```

{
    private readonly StatisticsService _statisticsService;

    // Backing fields и свойства – см. раздел 4.7

    public StatisticsWindow(StatisticsService statisticsService)
    {
        InitializeComponent();
        _statisticsService = statisticsService;
        DataContext = this;
        LoadStatistics();
    }

    private void LoadStatistics()
    {
        var filter = new ReservationsFilter
        {
            StartDate = StartDate,
            EndDate = EndDate
        };

        LoadStatusChart(filter);
        LoadMasterChart(filter);
        LoadMonthChart(filter);
    }

    private void ApplyFilterButton_Click(object sender, RoutedEventArgs e)
    {
        LoadStatistics();
    }

    private void ResetFilterButton_Click(object sender, RoutedEventArgs e)
    {
        StartDate = null;
        EndDate = null;
        LoadStatistics();
    }

    // LoadStatusChart, LoadMasterChart, LoadMonthChart, INotifyPropertyChanged – из раздела 4.7
}

```

Шаг 2: Добавить кнопку в главное окно

В главном окне приложения (например, `ReservationsListWindow.xaml`) добавьте кнопку:

```
<Button Content="Статистика" Width="120" Click="StatisticsButton_Click"/>
```

```

private void StatisticsButton_Click(object sender, RoutedEventArgs e)
{
    var statisticsService = new StatisticsService(_reservationRepository);
    var window = new StatisticsWindow(statisticsService);
    window.ShowDialog();
}

```

Шаг 3: Добавить тестовые данные

Зачем нужны тестовые данные:

- Для красивых диаграмм нужно минимум 20-50 записей

- Данные должны охватывать разные периоды времени (несколько месяцев)
- Данные должны быть распределены по разным категориям (enum, связанные сущности)

Что адаптировать под свою предметную область:

- Массивы значений enum
- Массивы связанных сущностей (создайте 3-5 объектов)
- Свойства создаваемых объектов
- Логику генерации дат (используйте `DateTime.Now.AddDays(-random.Next(0, 180))` для дат за последние 6 месяцев)

Общая структура метода `SeedData`:

1. Создать массивы значений enum (если есть)
2. Создать массивы связанных сущностей (если есть)
3. В цикле сгенерировать 30-50 тестовых записей с случайными данными

Пример для салона красоты:

```
public ReservationRepository()
{
    SeedData();
}

private void SeedData()
{
    var statuses = new[]
    {
        ReservationStatus.Created,
        ReservationStatus.Confirmed,
        ReservationStatus.Completed,
        ReservationStatus.Cancelled
    };

    var masters = new[]
    {
        new Master { Id = Guid.NewGuid(), Name = "Анна", Specialization = "Маникюр" },
        new Master { Id = Guid.NewGuid(), Name = "Мария", Specialization = "Стрижка" },
        new Master { Id = Guid.NewGuid(), Name = "Елена", Specialization = "Окрашивание" }
    };

    var random = new Random();

    for (int i = 0; i < 50; i++)
    {
        var reservation = new Reservation
        {
            Id = Guid.NewGuid(),
            Client = new Client
            {
                Name = $"Клиент {i + 1}",
                Phone = $"7900{random.Next(1000000, 9999999)}"
            },
            Master = masters[random.Next(masters.Length)],
            ServiceDescription = "Тестовая услуга",
            Date = DateOnly.FromDateTime(DateTime.Now.AddDays(-random.Next(0, 180))),
            StartTime = TimeOnly.FromTimeSpan(TimeSpan.FromHours(random.Next(9, 18))),
            Duration = TimeSpan.FromMinutes(30),
            Status = statuses[random.Next(statuses.Length)]
        };
    }
}
```

```
        _reservations.Add(reservation);  
    }  
}
```

4.9 Фиксация изменений

- Протестировать все расчёты статистики
- Проверить корректность отображения графиков при разных фильтрах
- Создать коммит:

```
git add .  
git commit -m "Добавление модуля статистики с визуализацией OxyPlot"
```

Ожидаемый результат

По итогам работы должно быть реализовано:

- **Архитектура:** Новый проект <ИмяПроекта>.Services с классом `StatisticsService` и корректными ссылками между проектами
- **Модели статистики:** 2-3 record-класса в папке `Domain/Statistics` для результатов расчётов
- **Фильтрация:** Класс <Сущность>Filter и обновлённый метод `GetAll` в репозитории
- **Расчёт метрик:** Методы в `StatisticsService` с использованием LINQ для группировки и агрегации данных
- **Визуализация:** Окно со статистикой, содержащее минимум 2 типа диаграмм (круговая, столбчатая или линейная)
- **Функциональность:** Работающие фильтры по датам, корректное отображение данных на графиках
- **Тестовые данные:** Много (предположительно, 30-50) записей в репозитории для демонстрации статистики
- **Git-коммит:** Зафиксированные изменения с описательным сообщением

Полезные ссылки

- Документация OxyPlot: <https://oxyplot.readthedocs.io/>
- GitHub OxyPlot: <https://github.com/oxyplot/oxyplot>